Inter-Waypoint Navigation

1. Course Content

Basic

- Start the road network navigation feature and learn to use the `roadnet_nav` visualization panel for waypoint-to-waypoint navigation

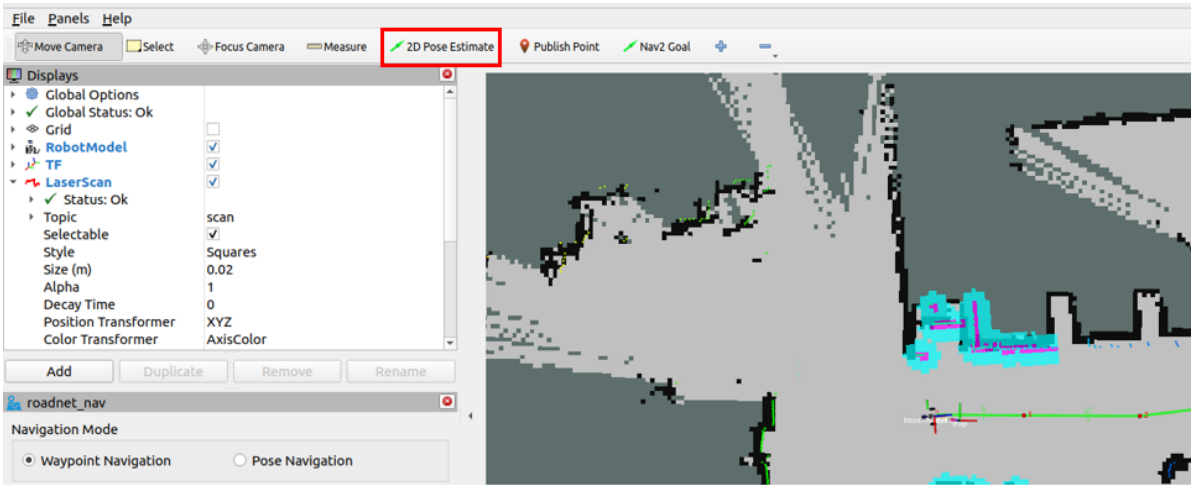
Advanced

- Learn how to request navigation through the action server interface
- Learn how to debug the navigation parameter file
- Understand the source code implementation logic

2. Preparation

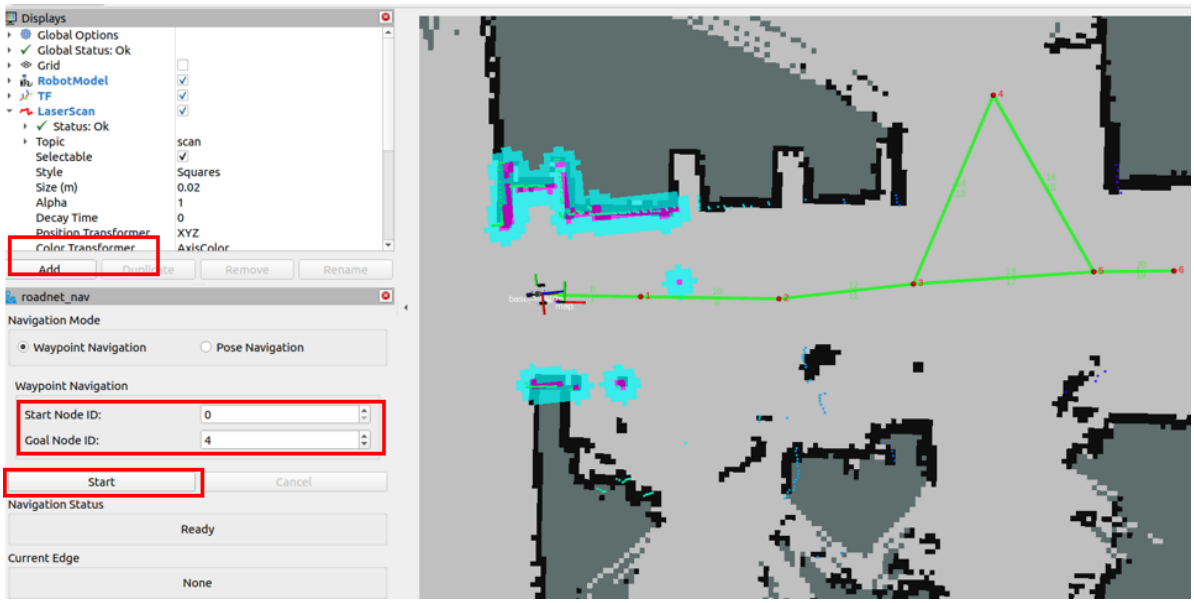
```
ros2 launch microros_v2_bringup roadnet_nav.launch.py
```

- Click **2D Pose Estimate** to initialize the robot's pose on the map



3. Waypoint-to-Waypoint Navigation

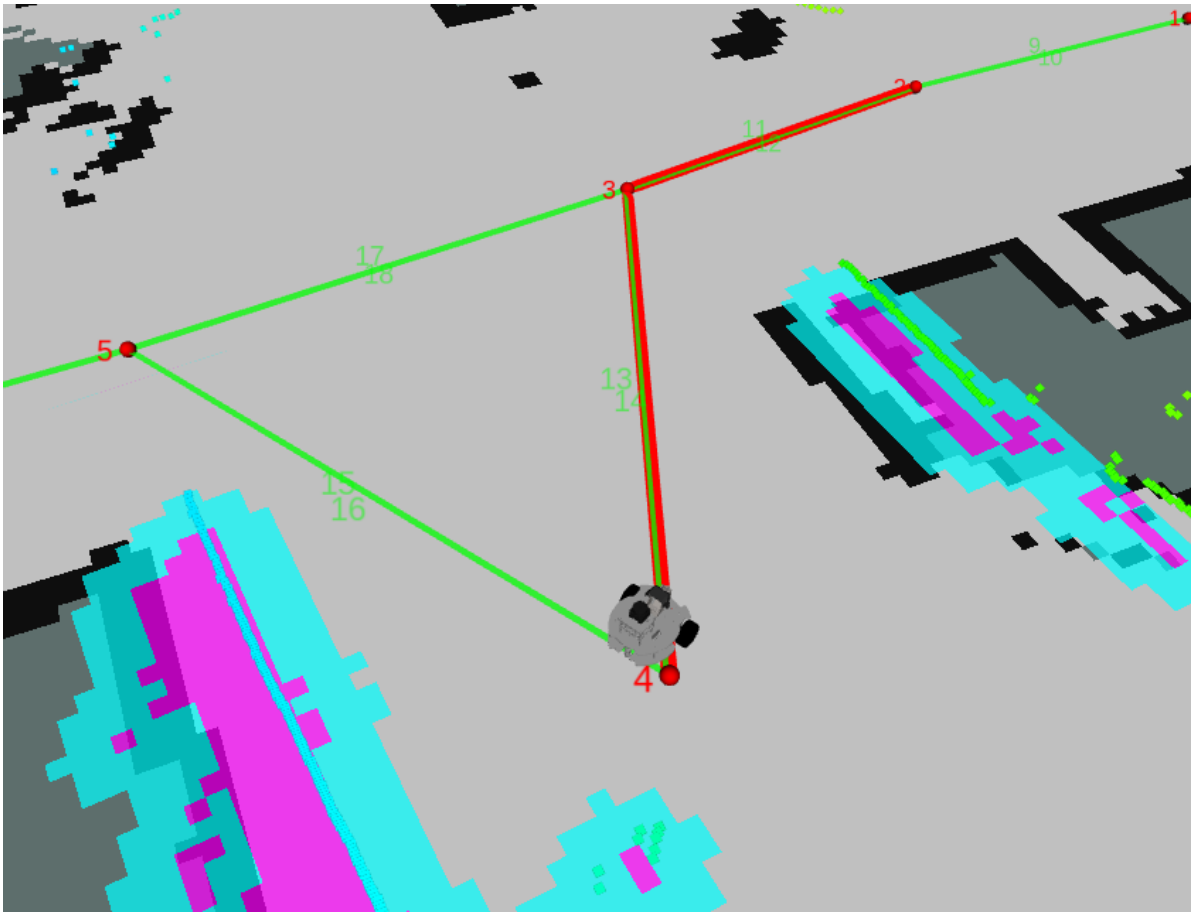
- Here we use the road network navigation through the `roadnet_nav` plugin interface. For example, in the figure our robot is close to waypoint 0, and we need to navigate to waypoint 4. Enter the **Start Node ID** and **Goal Node ID**, then click **Start**



- You can observe in rviz that the planned global path stays within the road network and follows the route

3.1 Navigating When the Starting Position Is Not on the Road Network

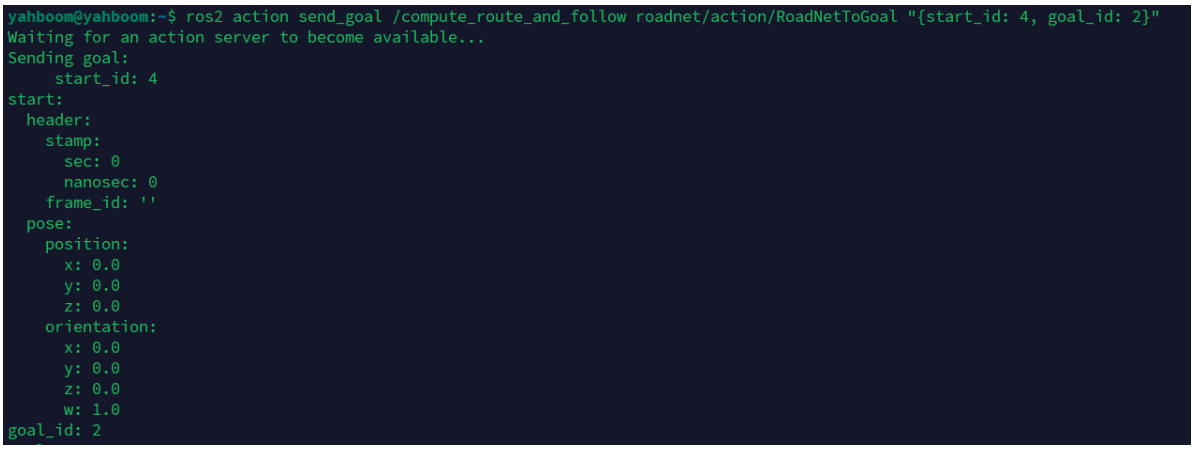
- If the starting position is not on a road network point or edge, when performing road network navigation, the robot will automatically first enter the nearest road network edge, and then move along the road network. For example, in the following situation, the robot is not on the road network. We observe that the nearest waypoint is 4. Here we plan to go to waypoint 2, entering the start and goal IDs



3.2 Requesting Road Network Navigation through the Action Server Interface

> [!TIP]
>
> - The action interface can be integrated into other programs for feature development

```
ros2 action send_goal /compute_route_and_follow roadnet/action/RoadNetToGoal "
{start_id: 4, goal_id: 2}"
```



4 Source Code Analysis

- Source code path of the implementation program:

```
$HOM/mircoros_ws/src/roadnet/src/route_server.cpp
```

(Explain the program here - to be added)

```
  void RouteServer::computeRouteAndFollow()
  {
    RCLCPP_INFO(get_logger(), "Computing route and following path to goal.");

    auto goal = compute_route_and_follow_server_->get_current_goal();
    auto result = std::make_shared<RoadNetToGoalResult>();
    auto feedback = std::make_shared<RoadNetToGoalFeedback>();
    ReroutingState rerouting_info;
    auto start_time = this->now();

    try
    {
      Route route;
      nav_msgs::msg::Path path;

      // Replanning trigger condition: (obstacle detection) or a replanning
request (ReroutingService)
      while (rclcpp::ok())
      {
        if (!isRequestValid(compute_route_and_follow_server_))
        {
          return;
```

```
        }
        // Check whether a new goal has preempted the current one
        if (compute_route_and_follow_server_->is_preempt_requested())
        {
          RCLCPP_INFO(get_logger(), "A new request is received, and the target
of the original request is preempted");
          goal = compute_route_and_follow_server_->accept_pending_goal();
          rerouting_info.reset();
        }
        // Clear residual cost values from the global_costmap before planning to
avoid affecting the plan
        clearGlobalCostmap();

        // Compute the feasible path in the road network
        route = findRoute(goal, rerouting_info);
        RCLCPP_INFO(get_logger(), "Route found with %zu nodes and %zu edges",
                    route.edges.size() + 1u, route.edges.size());
        path = path_converter_->densify(route, rerouting_info, route_frame_,
this->now());

        // Print the path
        std::string route_str;
        if (route.start_node)
        {
          route_str = std::to_string(route.start_node->nodeid);
        }
        for (const auto &edge : route.edges)
        {
          route_str += " ---------->> " + std::to_string(edge->end->nodeid);
        }
        RCLCPP_INFO(get_logger(), "Find route plan: %s", route_str.c_str());

        // Wait for FollowPath action server
        if (!follow_path_client_-
>wait_for_action_server(std::chrono::seconds(5)))
        {
          RCLCPP_ERROR(get_logger(), "FollowPath action server not available!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath action server not available";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }

        // Send the path to FollowPath
        auto follow_goal = FollowPath::Goal();
        follow_goal.path = path;

        auto send_goal_options =
rclcpp_action::Client<FollowPath>::SendGoalOptions();
        auto goal_handle_future = follow_path_client_-
>async_send_goal(follow_goal, send_goal_options);

        // Wait for goal acceptance
        if (goal_handle_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal send timed out!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
```

```
          result->error_msg = "FollowPath goal send timed out";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }

        auto follow_goal_handle = goal_handle_future.get();
        if (!follow_goal_handle)
        {
          RCLCPP_ERROR(get_logger(), "FollowPath goal was rejected!");
          result->error_code = RoadNetToGoalResult::UNKNOWN;
          result->error_msg = "FollowPath goal rejected by controller";
          compute_route_and_follow_server_->terminate_current(result);
          return;
        }
        RCLCPP_DEBUG(get_logger(), "FollowPath goal accepted, robot is
moving!");

        // Get result future for polling
        auto follow_result_future = follow_path_client_-
>async_get_result(follow_goal_handle);

        // Track path progress and detect obstacles
        double radius_threshold = 2.0;
        nav2_util::declare_parameter_if_not_declared(
            shared_from_this(), "radius_to_achieve_node",
            rclcpp::ParameterValue(radius_threshold));
        radius_threshold = get_parameter("radius_to_achieve_node").as_double();

        EdgePtr current_edge = nullptr;
        NodePtr next_node = nullptr;
        int edge_idx = -1;
        RouteTrackingState tracking_state;
        tracking_state.last_node = route.start_node;
        tracking_state.next_node = route.start_node;

        // Advance to first edge
        if (!route.edges.empty())
        {
          edge_idx = 0;
          current_edge = route.edges[0];
          next_node = current_edge->end;
          tracking_state.current_edge = current_edge;
          RCLCPP_INFO(get_logger(),
                      "Entering edge:%u  id:%u ----------------------> id:%u",
                      current_edge->edgeid,
                      current_edge->start->nodeid,
                      current_edge->end->nodeid);
        }
        rclcpp::Rate rate(20.0);
        bool follow_completed = false;
        bool route_tracking_done = route.edges.empty();
        bool need_reroute = false;

        while (rclcpp::ok() && !follow_completed && !need_reroute)
        {
          // Check if our action was cancelled
          if (compute_route_and_follow_server_->is_cancel_requested())
          {
```

```
            RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling
FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            compute_route_and_follow_server_->terminate_all();
            return;
          }

          // Check if preempted by a new goal
          if (compute_route_and_follow_server_->is_preempt_requested())
          {
            RCLCPP_INFO(get_logger(), "Goal preempted, cancelling
FollowPath...");
            follow_path_client_->async_cancel_goal(follow_goal_handle);
            break;  // Break out of the inner loop; the outer loop will handle
the new goal
          }

          // Check if FollowPath completed
          if (follow_result_future.wait_for(std::chrono::milliseconds(0)) ==
              std::future_status::ready)
          {
            auto wrapped_result = follow_result_future.get();
            if (wrapped_result.code == rclcpp_action::ResultCode::SUCCEEDED)
            {
              RCLCPP_INFO(get_logger(), "FollowPath completed successfully!");
            }
            else if (wrapped_result.code == rclcpp_action::ResultCode::CANCELED)
            {
              RCLCPP_WARN(get_logger(), "FollowPath was cancelled.");
            }
            else
            {
              RCLCPP_WARN(get_logger(), "FollowPath failed
(code=%d).",static_cast<int>(wrapped_result.code));
            }
            follow_completed = true;
            break;
          }

          // Get the robot's current position
          geometry_msgs::msg::PoseStamped robot_pose;
          bool got_pose = nav2_util::getCurrentPose(
              robot_pose, *tf_, route_frame_, base_frame_, 0.5);

          // Track edge transitions
          if (!route_tracking_done && next_node && got_pose)
          {
            const double dx = next_node->coords.x - robot_pose.pose.position.x;
            const double dy = next_node->coords.y - robot_pose.pose.position.y;
            const double dist = std::sqrt(dx * dx + dy * dy);

            if (dist < radius_threshold)
            {
              // Node achieved
              RCLCPP_INFO(get_logger(),
                          "Arrived at node id:%u (%.2f, %.2f)",
                          next_node->nodeid,
                          next_node->coords.x, next_node->coords.y);
```

```
              tracking_state.last_node = next_node;
              edge_idx++;

              if (edge_idx < static_cast<int>(route.edges.size()))
              {
                current_edge = route.edges[edge_idx];
                next_node = current_edge->end;
                tracking_state.current_edge = current_edge;
                tracking_state.next_node = next_node;
                tracking_state.route_edges_idx = edge_idx;

                RCLCPP_INFO(get_logger(),
                            "Entering edge:%u  id:%u ---------------------->
id:%u",
                            current_edge->edgeid,
                            current_edge->start->nodeid,
                            current_edge->end->nodeid);

                // Publish feedback on edge transition
                feedback->last_node_id = tracking_state.last_node ?
tracking_state.last_node->nodeid : 0;
                feedback->next_node_id = next_node ? next_node->nodeid : 0;
                feedback->current_edge_id = current_edge ? current_edge->edgeid
: 0;
                feedback->rerouted = false;
                compute_route_and_follow_server_->publish_feedback(feedback);
              }
              else
              {
                // All nodes achieved
                route_tracking_done = true;
                tracking_state.current_edge = nullptr;
                tracking_state.next_node = nullptr;
                RCLCPP_INFO(get_logger(), "All route nodes achieved!");
              }
            }
          }

          // Run OperationsManager for obstacle detection and replanning checks
          if (got_pose && operations_manager_)
          {
            bool status_change = (edge_idx >= 0 && tracking_state.last_node !=
nullptr);
            OperationsResult ops_result = operations_manager_->process(
                status_change, tracking_state, route, robot_pose,
rerouting_info);

            // If an operation triggered a replanning request
            if (ops_result.reroute)
            {
              RCLCPP_WARN(get_logger(), "Rerouting requested by operations
(e.g., collision detected)!");

              // Save the replanning information
              if (!ops_result.blocked_ids.empty())
              {
                if (!aggregate_blocked_ids_) {
```

```
                  rerouting_info.blocked_ids = ops_result.blocked_ids;
                } else {
                  rerouting_info.blocked_ids.insert(
                      rerouting_info.blocked_ids.end(),
                      ops_result.blocked_ids.begin(),
ops_result.blocked_ids.end());
                }
              }

              // Set the replanning start to the start node of the current
blocked edge so the robot backs up to a safe node and replans
              NodePtr reroute_start_node = nullptr;
              if (tracking_state.last_node)
              {
                reroute_start_node = tracking_state.last_node;
              }
              else if (tracking_state.current_edge)
              {
                reroute_start_node = tracking_state.current_edge->start;
              }

              if (reroute_start_node)
              {
                rerouting_info.rerouting_start_id = reroute_start_node->nodeid;
                rerouting_info.rerouting_start_pose.pose.position.x =
reroute_start_node->coords.x;
                rerouting_info.rerouting_start_pose.pose.position.y =
reroute_start_node->coords.y;
                rerouting_info.rerouting_start_pose.header.frame_id =
route_frame_;
                rerouting_info.rerouting_start_pose.header.stamp = this->now();
              }
              else
              {
                rerouting_info.rerouting_start_id = std::numeric_limits<unsigned
int>::max();
                rerouting_info.rerouting_start_pose =
geometry_msgs::msg::PoseStamped();
              }

              // Update so during rerouting we can check if we are continuing on
the same edge
              rerouting_info.curr_edge = tracking_state.current_edge;

              // Cancel the current FollowPath
              follow_path_client_->async_cancel_goal(follow_goal_handle);
              need_reroute = true;

              // Publish feedback with reroute info
              feedback->last_node_id = tracking_state.last_node ?
tracking_state.last_node->nodeid : 0;
              feedback->next_node_id = next_node ? next_node->nodeid : 0;
              feedback->current_edge_id = current_edge ? current_edge->edgeid :
0;
              feedback->operations_triggered = ops_result.operations_triggered;
              feedback->rerouted = true;
              compute_route_and_follow_server_->publish_feedback(feedback);
              break;
```

```
            }
          }

          rate.sleep();
        }

        // If replanning is needed, continue the outer loop
        if (need_reroute)
        {
          RCLCPP_INFO(get_logger(), "Rerouting from node %u (%.2f, %.2f)...",
              rerouting_info.rerouting_start_id,
              rerouting_info.rerouting_start_pose.pose.position.x,
              rerouting_info.rerouting_start_pose.pose.position.y);
          continue;
        }

        // If FollowPath completed or was cancelled, break out of the loop
        if (follow_completed || compute_route_and_follow_server_-
>is_cancel_requested())
        {
          break;
        }

        // If preempted, continue the outer loop to handle the new goal
        if (compute_route_and_follow_server_->is_preempt_requested())
        {
          continue;
        }
      }

      // Check whether it was cancelled
      if (compute_route_and_follow_server_->is_cancel_requested())
      {
        compute_route_and_follow_server_->terminate_all();
        return;
      }

      // If using pose navigation, navigate to goal pose after route following
      if (goal->use_poses)
      {
        RCLCPP_INFO(get_logger(), "Navigating to goal pose...");

        // Wait for NavigateToPose action server
        if (!nav_to_pose_client_-
>wait_for_action_server(std::chrono::seconds(5)))
        {
          RCLCPP_WARN(get_logger(), "NavigateToPose action server not available,
skipping precise navigation.");
        }
        else
        {
          // Create NavigateToPose goal
          auto nav_goal = NavigateToPose::Goal();
          nav_goal.pose = goal->goal;

          // Send goal to NavigateToPose
          auto nav_goal_options =
rclcpp_action::Client<NavigateToPose>::SendGoalOptions();
```

```
          auto nav_goal_future = nav_to_pose_client_->async_send_goal(nav_goal,
nav_goal_options);

          // Wait for goal acceptance
          if (nav_goal_future.wait_for(std::chrono::seconds(5)) !=
std::future_status::ready)
          {
            RCLCPP_WARN(get_logger(), "NavigateToPose goal send timed out,
skipping precise navigation.");
          }
          else
          {
            auto nav_goal_handle = nav_goal_future.get();
            if (!nav_goal_handle)
            {
              RCLCPP_WARN(get_logger(), "NavigateToPose goal was rejected,
skipping precise navigation.");
            }
            else
            {
              RCLCPP_INFO(get_logger(), "NavigateToPose goal accepted, robot is
moving to precise goal pose!");

              // Get result future for polling
              auto nav_result_future = nav_to_pose_client_-
>async_get_result(nav_goal_handle);

              // Wait for NavigateToPose to complete
              rclcpp::Rate nav_rate(20.0);
              bool nav_completed = false;

              while (rclcpp::ok() && !nav_completed)
              {
                // Check if our action was cancelled
                if (compute_route_and_follow_server_->is_cancel_requested())
                {
                  RCLCPP_INFO(get_logger(), "Goal cancelled, cancelling
NavigateToPose...");
                  nav_to_pose_client_->async_cancel_goal(nav_goal_handle);
                  compute_route_and_follow_server_->terminate_all();
                  return;
                }

                // Check if preempted by a new goal
                if (compute_route_and_follow_server_->is_preempt_requested())
                {
                  RCLCPP_INFO(get_logger(), "Goal preempted, cancelling
NavigateToPose...");
                  nav_to_pose_client_->async_cancel_goal(nav_goal_handle);
                  // Recursively handle new goal
                  computeRouteAndFollow();
                  return;
                }

                // Check if NavigateToPose completed
                if (nav_result_future.wait_for(std::chrono::milliseconds(0)) ==
                    std::future_status::ready)
                {
```

```
                auto nav_result = nav_result_future.get();
                if (nav_result.code == rclcpp_action::ResultCode::SUCCEEDED)
                {
                  RCLCPP_INFO(get_logger(), "NavigateToPose completed
successfully!");
                }
                else if (nav_result.code ==
rclcpp_action::ResultCode::CANCELED)
                {
                  RCLCPP_WARN(get_logger(), "NavigateToPose was cancelled.");
                }
                else
                {
                  RCLCPP_WARN(get_logger(), "NavigateToPose failed
(code=%d).",
                              static_cast<int>(nav_result.code));
                }
                nav_completed = true;
                break;
              }

              nav_rate.sleep();
            }
          }
        }
      }

      // Return result
      result->success = true;
      result->execution_duration = this->now() - start_time;
      RCLCPP_INFO(get_logger(), "========== Route and follow completed!
==========");
      compute_route_and_follow_server_->succeeded_current(result);
    }
    catch (nav2_core::NoValidRouteCouldBeFound &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::NO_VALID_ROUTE;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::TimedOut &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::TIMEOUT;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::RouteTFError &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::TF_ERROR;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::NoValidGraph &ex)
    {
```

```
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::NO_VALID_GRAPH;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (nav2_core::IndeterminantNodesOnGraph &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::INDETERMINANT_NODES_ON_GRAPH;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
    catch (std::exception &ex)
    {
      exceptionWarning(goal, ex);
      result->error_code = RoadNetToGoalResult::UNKNOWN;
      result->error_msg = ex.what();
      compute_route_and_follow_server_->terminate_current(result);
    }
  }cpp
```

5. Parameter Debugging

Path of the road network navigation parameter file:

```
$HOME/mircoros_ws/src/microros_v2_bringup/config/roadnet_nav.yaml
```

5.1 Navigation Controller Debugging

> [!TIP]
>
> **Parameter Adjustment Guide**
>
> - If you need to adjust the linear speed and rotational angular velocity during navigation, adjust `desired_linear_vel` and `max_angular_accel`
> - If you need to adjust the accuracy of reaching the goal point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
> - For other parameter adjustments, see the comments below

```yaml
controller_server:
  ros__parameters:
    # ====================== Controller core global parameters
======================
    # Main loop frequency of the controller (Hz); determines the publish rate of
local path planning and velocity commands, and must match the hardware execution
frequency
    controller_frequency: 20.0
    # Costmap update timeout (s); if the costmap is not updated within this time,
the controller stops publishing velocity commands to avoid planning based on an
outdated map
    costmap_update_timeout: 0.30
    # Minimum X-direction (forward/backward) velocity threshold (m/s); velocity
commands below this value are treated as invalid to filter out micro jitter
```

```yaml
    min_x_velocity_threshold: 0.001
    # Minimum Y-direction (lateral) velocity threshold (m/s); only applies to omnidirectional robots (e.g., Mecanum wheels). Differential-drive robots can set a larger value (e.g., 0.5) to disable lateral movement
    min_y_velocity_threshold: 0.5
    # Minimum rotation velocity threshold (rad/s); rotation commands below this value are treated as invalid to filter out tiny rotation jitter
    min_theta_velocity_threshold: 0.001
    # Controller failure tolerance (m); when the local path deviation exceeds this value, the controller is judged to have failed and replanning is triggered
    failure_tolerance: 0.3
    # List of progress checker plugins, used to monitor whether the robot is moving toward the goal
    progress_checker_plugins: ["progress_checker"]
    # List of goal checker plugins, used to determine whether the robot has reached the goal pose
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugins, the core of which is the local path tracking algorithm; here the pure pursuit controller corresponding to FollowPath is used
    controller_plugins: ["FollowPath"]
    # Whether to use real-time priority scheduling for the controller thread
    use_realtime_priority: false
    # Speed limit topic name; the controller subscribes to velocity commands on this topic to dynamically adjust the maximum linear/angular velocity (e.g., slow down when avoiding obstacles)
    speed_limit_topic: "speed_limit"

    # ====================== Progress checker parameters (SimpleProgressChecker) ======================
    progress_checker:
      # Progress checker plugin type; SimpleProgressChecker is the base implementation that monitors the robot's traveled distance and time
      plugin: "nav2_controller::SimpleProgressChecker"
      # Minimum distance threshold (m) the robot must move; if it does not move more than this distance within movement_time_allowance, it is judged as making no progress
      required_movement_radius: 0.05
      # Maximum allowed no-progress time (s); if required_movement_radius is still not satisfied after this time, the controller aborts the task (error code 105)
      movement_time_allowance: 10.0

    # ====================== Goal checker parameters (SimpleGoalChecker) ======================
    general_goal_checker:
      # Whether it is a stateful checker; when set to True it continuously monitors the goal pose until the tolerance is satisfied; when False it only checks once
      stateful: True
      # Goal checker plugin type; SimpleGoalChecker is the base implementation that determines whether position and angle are within tolerance
      plugin: "nav2_controller::SimpleGoalChecker"
      # XY-plane position tolerance (m); when the deviation between the robot position and the goal position is less than this value, the position is considered reached
      xy_goal_tolerance: 0.08
```

```
      # Heading angle tolerance (rad); when the deviation between the robot
heading and the goal heading is less than this value, the angle is considered
reached
      yaw_goal_tolerance: 0.03

    # ====================== Pure pursuit controller parameters
(RegulatedPurePursuitController) ======================
    FollowPath:
      # Local path tracking plugin type; RegulatedPurePursuitController is a
pure pursuit algorithm with velocity regulation, suitable for differential-drive
robots
      plugin:
"nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
      # Desired linear velocity (m/s); the target linear velocity during normal
driving, dynamically scaled by obstacle avoidance/curvature regulation
      desired_linear_vel: 0.2
      # Lookahead distance (m); the core parameter of the pure pursuit algorithm
that determines how far ahead on the path the robot tracks, and must match the
robot speed
      lookahead_dist: 0.5
      # Minimum lookahead distance (m); prevents the robot from turning too
frequently when the lookahead distance is too small
      min_lookahead_dist: 0.05
      # Maximum lookahead distance (m); prevents the robot from reacting
sluggishly when the lookahead distance is too large
      max_lookahead_dist: 0.9
      # Lookahead time (s); if use_velocity_scaled_lookahead_dist is enabled,
lookahead distance = desired_linear_vel * lookahead_time
      lookahead_time: 1.5
      # Angular velocity for rotating to the goal heading (rad/s); the rotation
speed to adjust the heading after the robot reaches the position
      rotate_to_heading_angular_vel: 1.3
      # TF transform tolerance (s); the maximum allowed deviation between the
timestamp of the robot pose transform and the current time, solving TF delay
issues
      transform_tolerance: 0.5
      # Whether to use a velocity-scaled lookahead distance; when set to True,
the lookahead distance changes dynamically with robot speed (the faster the
speed, the farther the lookahead)
      use_velocity_scaled_lookahead_dist: false
      # Minimum linear velocity when approaching the goal (m/s); when the robot
is close to the goal, it slows down to this value to avoid overshooting the goal
      min_approach_linear_velocity: 0.05
      # Approach velocity scaling distance (m); when the robot is closer to the
goal than this value, it starts decelerating linearly (from desired_linear_vel to
min_approach_linear_velocity)
      approach_velocity_scaling_dist: 1.0
      # Whether to enable collision detection; when set to True, the controller
checks for collisions along the lookahead path to avoid the robot hitting
obstacles
      use_collision_detection: false
      # Maximum allowed time to collision up to the carrot point (s); only takes
effect when use_collision_detection is True, used to assess collision risk
      max_allowed_time_to_collision_up_to_carrot: 1.0
      # Whether to enable path-curvature-based linear velocity regulation; when
set to True, the smaller the turning radius, the lower the linear velocity
      use_regulated_linear_velocity_scaling: true
```

```
      # Whether to enable costmap-based linear velocity regulation; when set to
True, the closer the obstacle, the lower the linear velocity
      use_cost_regulated_linear_velocity_scaling: false
      # Minimum turning radius (m) that triggers curvature regulation; when the
turning radius corresponding to the path curvature is smaller than this value,
deceleration starts
      regulated_linear_scaling_min_radius: 0.05
      # Minimum speed for curvature regulation (m/s); even with an extremely
small turning radius, the linear velocity will not fall below this value
      regulated_linear_scaling_min_speed: 0.25
      # Whether to use a fixed-curvature lookahead distance; when set to True,
the lookahead distance is determined by curvature_lookahead_dist
      use_fixed_curvature_lookahead: false
      # Curvature lookahead distance (m); only takes effect when
use_fixed_curvature_lookahead is True
      curvature_lookahead_dist: 1.0
      # Whether to enable “rotate to heading before moving”; when set to True,
if the robot heading deviates too much from the path direction, it rotates first
and then moves forward
      use_rotate_to_heading: true
      # Minimum angle (rad) that triggers rotation to heading; when the
deviation between the robot heading and the path direction exceeds this value,
rotation adjustment starts
      rotate_to_heading_min_angle: 0.785 # about 45 degrees
      # Maximum angular acceleration (rad/s^2); limits the robot's rotational
acceleration to avoid abrupt turns damaging the hardware
      max_angular_accel: 1.2
      # Maximum distance (m) for searching the robot pose; when the controller
cannot find a matching robot pose on the path, it searches path points within
this distance
      max_robot_pose_search_dist: 10.0
      # Whether to interpolate curvature after the goal; when set to True,
curvature is interpolated on the path after the goal to prevent the robot from
stopping abruptly after reaching the goal
      interpolate_curvature_after_goal: false
      # Cost scaling distance (m); only takes effect when
use_cost_regulated_linear_velocity_scaling is True, used to compute the weight of
obstacle-avoidance speed reduction
      cost_scaling_dist: 0.08
      # Cost scaling gain; only takes effect when
use_cost_regulated_linear_velocity_scaling is True; the larger the gain, the more
pronounced the obstacle-avoidance speed reduction
      cost_scaling_gain: 1.0
      # Inflation cost scaling factor; converts the inflation layer cost of the
costmap into a velocity scaling coefficient; the larger the value, the more
sensitive the avoidance of inflated areas
      inflation_cost_scaling_factor: 3.0
```

5.2 Obstacle Avoidance and Costmap Debugging

> [!TIP]
>
> - If the robot navigates too close to obstacles in the map, you can keep the robot away from obstacles by adjusting the inflation area: increase the `inflation_radius` (inflation radius) and `cost_scaling_factor` (scaling factor; the larger the value, the

less likely the robot is to approach obstacles) in both `local_costmap` and `global_costmap`

- For other parameter adjustments, see the comments below

```yaml
# Local costmap configuration: a short-term costmap for real-time obstacle avoidance, rolling and updating as the robot moves
local_costmap:
  local_costmap:
    ros__parameters:
      # ====================== Costmap core global parameters ======================
      # Costmap update frequency (Hz); determines the refresh rate of the local map and must match the sensor frequency
      update_frequency: 5.0
      # Costmap publish frequency (Hz); the frequency at which the map is published to visualization tools such as RViz, lower than the update frequency to reduce bandwidth usage
      publish_frequency: 2.0
      # Global coordinate frame of the costmap; the local map uses the odom frame
      global_frame: odom
      # Robot base frame, consistent with the robot center frame in the hardware configuration
      robot_base_frame: base_link
      # Whether to enable rolling window mode, the core feature of the local map: the map rolls as the robot moves, keeping only the map around the robot
      rolling_window: true
      # Rolling window width (m); the lateral range of the local map
      width: 3
      # Rolling window height (m); the longitudinal range of the local map
      height: 3
      # Costmap resolution (m/pixel); the actual distance represented by each pixel; 0.05 means 5 cm/pixel
      resolution: 0.05
      # Robot footprint (polygon); represents the robot's actual shape as a list of coordinates
      footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
      # Robot footprint padding (m); an extra safety distance beyond the actual footprint, used as a buffer for obstacle avoidance
      footprint_padding: 0.0616
      # Costmap layer plugin list, loaded in order (voxel_layer: voxel obstacle layer; inflation_layer: inflation layer)
      plugins: ["voxel_layer", "inflation_layer"]
      # Costmap filter plugin list, used to post-process the map (keepout_filter: keepout filtering)
      filters: ["keepout_filter"]
      # Whether to always publish the full costmap; when set to True, the full map is published on every update; when False, only changed regions are published (reducing bandwidth)
      always_send_full_costmap: True
      # Service introspection mode; disabled means service introspection is disabled; options are enabled/verbose
      service_introspection_mode: "disabled"
```

```yaml
      # ====================== Filter plugin: keepout filtering (KeepoutFilter)
======================
      keepout_filter:
        # Filter plugin type; KeepoutFilter is used to mark keepout zones in the
costmap
        plugin: "nav2_costmap_2d::KeepoutFilter"
        # Whether to enable keepout filtering; KEEPOUT_ZONE_ENABLED is an
external parameter (must be assigned true/false at startup)
        enabled: KEEPOUT_ZONE_ENABLED
        # Subscription topic for keepout zone information; receives the geometry
and position of the keepout zones (e.g., PolygonStamped)
        filter_info_topic: "keepout_costmap_filter_info"
        # Whether to override the lethal cost of keepout zones; when set to
True, a custom cost replaces the default lethal cost (255)
        override_lethal_cost: True
        # Custom lethal cost for keepout zones; 200 is lower than the default
lethal cost (255) but is still a high-cost keepout zone
        lethal_override_cost: 200

      # ====================== Layer plugin: inflation layer (InflationLayer)
======================
      inflation_layer:
        # Layer plugin type; InflationLayer generates inflated zones around
obstacles to prevent robot collisions
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor; controls the cost decay rate in the inflated zone
(the larger the value, the faster the decay, the smaller the inflated zone)
        cost_scaling_factor: 3.0
        # Inflation radius (m); the inflated range around obstacles, the minimum
safe distance from the robot footprint to obstacles
        inflation_radius: 0.1

# Global costmap configuration: a long-term costmap for global path planning,
covering the entire environment
global_costmap:
  global_costmap:
    ros__parameters:
      # ====================== Costmap core global parameters
======================
      # Costmap update frequency (Hz); the global map does not need high-
frequency updates; 1-2 Hz is sufficient
      update_frequency: 1.0
      # Costmap publish frequency (Hz); can be consistent with the update
frequency
      publish_frequency: 1.0
      # Global coordinate frame of the costmap; the global map uses the map
frame (globally consistent, suitable for path planning)
      global_frame: map
      # Robot base frame, consistent with the local map
      robot_base_frame: base_link
      # Robot radius (replaced by footprint, commented out)
      # robot_radius: 0.22
      # Robot footprint (consistent with the local map)
      footprint: "[[0.14,0.11], [0.14,-0.11], [-0.14,-0.11], [-0.14,0.11]]"
      # Robot footprint padding (consistent with the local map)
      footprint_padding: 0.0616
      # Costmap resolution (consistent with the local map to ensure coordinate
matching)
```

```
      resolution: 0.05
      # Whether to track unknown space; when set to True, unexplored areas are
marked on the map (used for SLAM and planning in unknown environments)
      track_unknown_space: true
      # Costmap layer plugin list (static_layer: static layer; obstacle_layer:
2D obstacle layer; inflation_layer: inflation layer)
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      # Costmap filter plugin list (keepout_filter: keepout filtering;
speed_filter: speed limit filtering)
      filters: ["keepout_filter", "speed_filter"]
      # Whether to always publish the full costmap
      always_send_full_costmap: True
      # Service introspection mode
      service_introspection_mode: "disabled"

      inflation_layer:
        # Layer plugin type; InflationLayer generates inflated zones around
obstacles to prevent robot collisions
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor; controls the cost decay rate in the inflated zone
(the larger the value, the faster the decay, the smaller the inflated zone)
        cost_scaling_factor: 3.0
        # Inflation radius (m); the inflated range around obstacles, the minimum
safe distance from the robot footprint to obstacles
        inflation_radius: 0.1
```